

""""

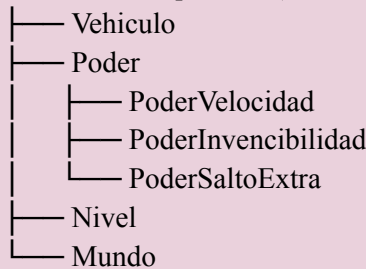
LÍNEA DE FUEGO - Videojuego de carreras en consola (Python)

Este programa implementa el videojuego descrito en los documentos:

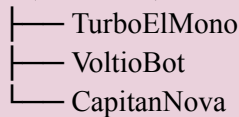
- Analisis.pdf -> mecánicas, mundos, niveles, premios
- Requerimientos.pdf -> requerimientos funcionales y no funcionales
- LÍNEA DE FUEGO.pdf -> diagrama de flujo y herencia POO

Estructura de clases (Programación Orientada a Objetos - Herencia):

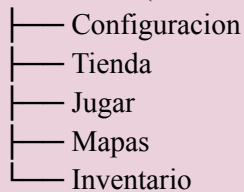
ElementoDesbloqueable (clase base general)



Jefe (clase base)



PantallaMenu (clase base)



Otras clases de apoyo: Herramienta, Mapa, Jugador,
ToggleConfiguracion, DecisionConfirmacion, ResultadoPartida.

El menú del juego sigue el DIAGRAMA DE FLUJO:

Línea de Fuego -> Registrarse -> Pantalla de Inicio -> Menú

Menú -> (Jugar | Configuración | Tienda | Ver Inventario | Salir)

Jugar -> Mapas -> Mundo 1/2/3 -> Niveles -> (Ganaste/Perdiste) -> Premio

""""

```
import random
import time
```

```
# =====
# 1. MANEJO DE ERRORES (Requerimiento no funcional: Robustez)
# =====
```

```
# Excepciones propias para que el juego nunca se cierre inesperadamente.
```

```
class ErrorLineaDeFuego(Exception):  
    """Excepción base de todos los errores propios del juego."""  
    pass
```

```
class ArchivoSonidoDanadoError(ErrorLineaDeFuego):  
    """Se lanza cuando un efecto de sonido no puede reproducirse."""  
    pass
```

```
class ComandoInvalidoError(ErrorLineaDeFuego):  
    """Se lanza cuando el usuario ingresa una opción de menú inválida."""  
    pass
```

```
def reproducir_sonido(nombre_efecto):  
    """  
    Simula la carga/reproducción de un efecto de sonido.  
    Si el archivo está 'dañado' (simulado al azar de forma muy poco  
    probable) el error se atrapa y el juego continúa sin congelarse.  
    """  
    try:  
        archivos_danados = [] # aquí se registrarían nombres de archivos corruptos  
        if nombre_efecto in archivos_danados:  
            raise ArchivoSonidoDanadoError(f"El archivo de sonido '{nombre_efecto}' está dañado.")  
        print(f" 🎧 [Sonido]: {nombre_efecto}")  
    except ArchivoSonidoDanadoError as error:  
        print(f" ⚠️ Aviso: {error}. Continuando sin sonido...")
```

```
def pedir_opcion(mensaje, opciones_validas):  
    """  
    Pide un comando al usuario y valida que sea una opción existente.  
    Atrapa comandos inválidos sin cerrar el programa (requerimiento  
    de robustez y tolerancia a fallos).  
    """  
    while True:  
        try:  
            entrada = input(mensaje).strip()  
            if entrada not in opciones_validas:  
                raise ComandoInvalidoError(f"'{entrada}' no es una opción válida.")  
            return entrada  
        except ComandoInvalidoError as error:  
            print(f" ⚠️ {error} Intenta de nuevo.")  
        except (KeyboardInterrupt, EOFError):  
            print("\n ⚠️ Entrada interrumpida. Volviendo al menú...")
```

```
return None
```

```
# =====  
# 2. CLASE BASE GENERAL: ElementoDesbloqueable  
# =====
```

```
# Patrón "bloqueado/desbloqueado" común a Vehículo, Poder, Herramienta,  
# Nivel y Mundo (jerarquía de herencia de dos niveles).
```

```
class ElementoDesbloqueable:  
    def __init__(self, nombre, bloqueado=True):  
        self.nombre = nombre  
        self.bloqueado = bloqueado  
  
    def desbloquear(self):  
        if self.bloqueado:  
            self.bloqueado = False  
            print(f"🔓 ¡{self.nombre} desbloqueado!")  
  
    def esta_bloqueado(self):  
        return self.bloqueado
```

```
# =====  
# 3. VEHÍCULOS  
# =====
```

```
class Vehiculo(ElementoDesbloqueable):  
    def __init__(self, nombre, velocidad, precio, habilidad_especial="Ninguna", bloqueado=True):  
        super().__init__(nombre, bloqueado)  
        self.velocidad = velocidad # atributo común  
        self.precio = precio # atributo común (Monedas Python)  
        self.habilidad_especial = habilidad_especial # atributo propio de cada vehículo  
  
    def __str__(self):  
        estado = "🔒 Bloqueado" if self.bloqueado else "✅ Disponible"  
        return f"{self.nombre} | Vel: {self.velocidad} | {self.precio} monedas | {estado}"
```

```
# Lista de 10 vehículos de la tienda (VEHÍCULO 1 ... VEHÍCULO 10 del diagrama)
```

```
def crear_vehiculos():  
    datos = [  
        ("Aleta de Tiburón", 60, 0, "Sin habilidad (inicial)"),  
        ("Buggy Selvático", 65, 300, "Mejor agarre en curvas"),  
        ("Neón Runner", 70, 600, "Estela de neón (distráe rivales)"),  
        ("Cromado Veloz", 75, 900, "Reflejo de luz (ceguera breve al rival)"),  
        ("Halcón de Asfalto", 78, 1200, "Frenado corto"),  
        ("Cohete Urbano", 82, 1500, "Turbo de arranque"),
```

```

("Deslizador Cósmico", 85, 1800, "Levita sobre obstáculos bajos"),
("Meteoro Plateado", 88, 2100, "Resistente a choques"),
("Astronauta Dorado", 95, 2500, "Turbo infinito breve (skin secreta)"),
("Prototipo Alfa", 99, 5000, "Todas las mejoras combinadas"),
]
vehiculos = []
for i, (nombre, velocidad, precio, habilidad) in enumerate(datos):
    # El primer vehículo viene desbloqueado desde el inicio
    vehiculos.append(Vehiculo(nombre, velocidad, precio, habilidad, bloqueado=(i != 0)))
return vehiculos

```

```

# =====
# 4. PODERES (Herencia: cada hija sobrescribe aplicar_efecto())
# =====

```

```

class Poder(ElementoDesbloqueable):
    def __init__(self, nombre, duracion, costo, bloqueado=True):
        super().__init__(nombre, bloqueado)
        self.duracion = duracion # atributo común
        self.costo = costo      # atributo común

    def activar(self):
        """Método común: valida el estado y delega en aplicar_efecto()."""
        if self.bloqueado:
            print(f"🔒 {self.nombre} está bloqueado. Cómpralo en la tienda.")
            return
        reproducir_sonido(f"activar_{self.nombre}")
        self.aplicar_efecto()

    def aplicar_efecto(self):
        # Método genérico; las subclases lo sobrescriben (polimorfismo)
        print(f"✨ {self.nombre} activado por {self.duracion}s.")

```

```

class PoderVelocidad(Poder):
    def aplicar_efecto(self):
        print(f"🚀 {self.nombre}: ¡velocidad aumentada un 50% durante {self.duracion}s!")

```

```

class PoderInvencibilidad(Poder):
    def aplicar_efecto(self):
        print(f"🛡️ {self.nombre}: invencibilidad activa durante {self.duracion}s.")

```

```

class PoderSaltoExtra(Poder):
    def aplicar_efecto(self):
        print(f"🦘 {self.nombre}: ¡salto extra habilitado por {self.duracion}s!")

```

```
def crear_poderes():
    clases = [PoderVelocidad, PoderInvencibilidad, PoderSaltoExtra]
    poderes = []
    for i in range(1, 13): # PODER 1 ... PODER 12
        clase = clases[i % len(clases)]
        poderes.append(clase(f'Poder {i}', duracion=3 + (i % 5), costo=150 * i))
    return poderes
```

```
# =====
# 5. HERRAMIENTAS (datos inmutables con TUPLAS)
# =====
```

```
class Herramienta:
```

```
    """
```

```
    Los datos fijos (nombre, duración) se guardan en una tupla para
    garantizar que sean inmutables durante la partida, tal como pide
    el requerimiento de "Integridad de Datos".
```

```
    """
```

```
def __init__(self, datos_fijos: tuple, disponible=True):
    self._datos = datos_fijos # tupla inmutable: (nombre, duracion_segundos)
    self.disponible = disponible
```

```
@property
```

```
def nombre(self):
    return self._datos[0]
```

```
@property
```

```
def duracion(self):
    return self._datos[1]
```

```
def usar(self, jugador):
```

```
    """Activa el poder de la herramienta llamando a una función
    independiente (def) según su identidad."""
```

```
    if not self.disponible:
```

```
        print(f" 🚫 No tienes {self.nombre} en el inventario.")
        return
```

```
    reproducir_sonido(f'usar_{self.nombre}')
```

```
    accion = ACCIONES_HERRAMIENTAS.get(self.nombre, accion_generica)
```

```
    accion(jugador, self.duracion)
```

```
# Datos inmutables (tuplas) de las 3 herramientas principales del análisis
```

```
DATOS_NITRO = ("Nitro Boost", 3)
```

```
DATOS_ESCUDO = ("Escudo de Plasma", 5)
```

```
DATOS_MISIL = ("Misil Teledirigido", 2)
```

```
# Funciones independientes (def) para cada herramienta, activadas al presionar un botón
```

```
def accion_nitro(jugador, duracion):
```

```
    jugador.velocidad_bonus += 0.5
```

```
    print(f" 🚀 Nitro Boost: +50% de velocidad durante {duracion}s.")
```

```
def accion_escudo(jugador, duracion):
```

```
    jugador.escudo_activo = True
```

```
    print(f" 🛡 Escudo de Plasma: protegido contra el próximo choque ({duracion}s).")
```

```
def accion_misil(jugador, duracion):
```

```
    print(f" 🎯 Misil Teledirigido: el rival queda frenado {duracion}s.")
```

```
def accion_generica(jugador, duracion):
```

```
    print(" ⚙ Herramienta usada.")
```

```
ACCIONES_HERRAMIENTAS = {
```

```
    "Nitro Boost": accion_nitro,
```

```
    "Escudo de Plasma": accion_escudo,
```

```
    "Misil Teledirigido": accion_misil,
```

```
}
```

```
def crear_herramientas_tienda():
```

```
    """Genera las 8 herramientas de la tienda (HERRAMIENTA 1..8)."""
```

```
    base = [DATOS_NITRO, DATOS_ESCUDO, DATOS_MISIL]
```

```
    herramientas = []
```

```
    for i in range(8):
```

```
        datos = base[i % len(base)]
```

```
        nombre_mostrado = (datos[0] if i < 3 else f'{datos[0]} MK{i // 3 + 1}')
```

```
        herramientas.append(Herramienta((nombre_mostrado, datos[1]), disponible=(i == 0)))
```

```
    return herramientas
```

```
# =====  
# 6. JEFES DE CARRERA (Inteligencia Artificial)  
# =====
```

```
class Jefe:
```

```
    def __init__(self, nombre, vehiculo_descripcion):
```

```
        self.nombre = nombre
```

```
        self.vehiculo_descripcion = vehiculo_descripcion
```

```
def ataque_unico(self, jugador):
    """Método a sobrescribir por cada jefe (polimorfismo)."""
    raise NotImplementedError
```

```
def __str__(self):
    return f'{self.nombre} ({self.vehiculo_descripcion})'
```

```
class TurboElMono(Jefe):
```

```
    def __init__(self):
        super().__init__("Turbo el Mono", "buggy de madera")
```

```
    def ataque_unico(self, jugador):
        if jugador.escudo_activo:
            print("🛡️ ¡El escudo bloqueó el plátano de Turbo el Mono!")
            jugador.escudo_activo = False
            return
        print("🍌 Turbo el Mono lanza un plátano... ¡resbalas y pierdes tiempo!")
        jugador.progreso -= 5
```

```
class VoltioBot(Jefe):
```

```
    def __init__(self):
        super().__init__("Voltio Bot", "auto flotante DJ")
```

```
    def ataque_unico(self, jugador):
        if jugador.escudo_activo:
            print("🛡️ ¡El escudo bloqueó el campo electromagnético de Voltio Bot!")
            jugador.escudo_activo = False
            return
        print("⚡ Voltio Bot activa un campo electromagnético... ¡te frenas!")
        jugador.progreso -= 4
```

```
class CapitanNova(Jefe):
```

```
    def __init__(self):
        super().__init__("Capitán Nova", "nave monoplaça")
```

```
    def ataque_unico(self, jugador):
        if jugador.escudo_activo:
            print("🛡️ ¡El escudo bloqueó el ataque de Capitán Nova!")
            jugador.escudo_activo = False
            return
        print("👁️ Capitán Nova usa un portal de velocidad y te adelanta agresivamente.")
        jugador.progreso -= 6
```

```
# =====  
# 7. NIVELES  
# =====
```

```
class Nivel(ElementoDesbloqueable):  
    def __init__(self, nombre, dificultad, descripcion, puntaje_minimo,  
                  premio_monedas, premio_item, atajo, bloqueado=True):  
        super().__init__(nombre, bloqueado)  
        self.dificultad = dificultad  
        self.descripcion = descripcion  
        self.puntaje_minimo = puntaje_minimo  
        self.premio_monedas = premio_monedas  
        self.premio_item = premio_item  
        self.atajo = atajo  
        self.completado = False  
  
    def jugar(self, jugador, jefe):  
        """  
        Simula la carrera con un ciclo WHILE (requerimiento del análisis:  
        'ciclo while para mantener la carrera activa hasta que el  
        jugador o el rival crucen la línea de meta').  
        """  
  
        print(f"\n=== 🏁 {self.nombre} (Dificultad {self.dificultad}/10) ===")  
        print(f"    {self.descripcion}")  
        print(f"    Atajo disponible: {self.atajo}")  
  
        jugador.progreso = 0  
        jugador.escudo_activo = False  
        jugador.velocidad_bonus = 0  
        rival_progreso = 0  
        meta = 100  
  
        while jugador.progreso < meta and rival_progreso < meta:  
            avance_jugador = random.randint(5, 12) + int(jugador.velocidad_bonus * 10)  
            avance_rival = random.randint(4, 10) + self.dificultad # el rival es más rápido en niveles  
dificiles  
            jugador.progreso += avance_jugador  
            rival_progreso += avance_rival  
            jugador.velocidad_bonus = max(0, jugador.velocidad_bonus - 0.15)  
  
            # Uso de atajo aleatorio (zona oculta detectada)  
            if random.random() < 0.15:  
                print(f"    ✨ ¡Encontraste el atajo! ({self.atajo})")  
                jugador.progreso += 8  
  
            # El jefe ataca de forma intermitente (condicional anidada)  
            if random.random() < 0.25:  
                jefe.ataque_unico(jugador)
```

```
        print(f"  Tú: {min(jugador.progreso, meta)}% | {jefe.nombre}: {min(rival_progreso, meta)}%")
```

```
    if jugador.progreso >= meta and jugador.progreso >= rival_progreso:
        return self.ganar(jugador)
    else:
        return self.perder()
```

```
def ganar(self, jugador):
    print(f"  🏆 ¡GANASTE el nivel {self.nombre}!")
    self.completado = True
    jugador.monedas += self.premio_monedas
    jugador.inventario_cosmeticos.append(self.premio_item)
    jugador.puntaje_total += self.dificultad * 10
    return True
```

```
def perder(self):
    print(f"  💣 PERDISTE. El nivel se reinicia (opción REINICIAR en el menú del nivel).")
    return False
```

```
def crear_niveles():
```

```
    """
```

Requerimiento funcional: 'lista de 6 niveles ordenados' que el jugador recorre por índice (0 al 5), desbloqueando el siguiente según puntaje mínimo y estado del nivel anterior (condicionales anidadas, ver desbloquear_niveles()).

```
    """
```

```
    datos = [
        ("Playa Alegre", 2, "Curvas anchas, sin obstáculos. Ideal para aprender el control.",
         0, 100, "Carrocería 'Aleta de Tiburón'", "Conducir detrás de la gran cascada."),
        ("Selva Enredada", 4, "Obstáculos fijos (troncos) y curvas cerradas.",
         50, 200, "Neumáticos de Agarre Selvático", "Ninguno registrado en este tramo."),
        ("Neón City", 6, "Tráfico de ciudadanos robóticos que esquivar a alta velocidad.",
         120, 400, "Motor Eléctrico Neón V1", "Saltar la rampa de basura tecnológica sube a un riel superior."),
        ("Autopista Elevada", 7, "Carriles estrechos y zonas sin barreras donde puedes caer.",
         200, 600, "Pintura Cromada Cambia-Color", "Ninguno registrado en este tramo."),
        ("Gravedad Cero", 8, "Pistas flotantes magnéticas, saltos con rampas obligatorias.",
         300, 1000, "Alerón Antigraavedad", "Ninguno registrado en este tramo."),
        ("Agujero Negro", 10, "Meteoritos caen dinámicamente; el rival usa turbos infinitos.",
         420, 2500, "Skin Secreta 'Astronauta Dorado' (Trofeo Final)",
         "Portal azul oculto detrás del segundo meteorito: teletransporta a la meta."),
    ]
    niveles = []
    for i, (nombre, dificultad, descripcion, puntaje_min, monedas, item, atajo) in enumerate(datos):
        niveles.append(Nivel(nombre, dificultad, descripcion, puntaje_min, monedas, item, atajo,
```

```
        bloqueado=(i != 0))) # el índice 0 siempre empieza desbloqueado
return niveles
```

```
def desbloquear_siguiente_nivel(niveles, indice_actual, jugador):
    """
    Condicionales anidadas: verifica si el jugador puede desbloquear
    el siguiente nivel según su puntaje y el estado del nivel anterior.
    """
    if indice_actual + 1 < len(niveles):
        siguiente = niveles[indice_actual + 1]
        anterior = niveles[indice_actual]
        if anterior.completado:
            if jugador.puntaje_total >= siguiente.puntaje_minimo:
                siguiente.desbloquear()
            else:
                print(f"    ¡ Necesitas {siguiente.puntaje_minimo} puntos para desbloquear "
                      f"{siguiente.nombre}' (tienes {jugador.puntaje_total}).")
```

```
# =====
# 8. MUNDOS Y MAPAS
# =====
```

```
class Mundo(ElementoDesbloqueable):
    def __init__(self, nombre, niveles, jefe, bloqueado=True):
        super().__init__(nombre, bloqueado)
        self.niveles = niveles      # lista de niveles de este mundo
        self.jefe = jefe

    def verificar_desbloqueo(self, jugador):
        """Un mundo se desbloquea si todos sus niveles previos están completos."""
        if all(nivel.completado for nivel in self.niveles):
            self.desbloquear()
```

```
class Mapa:
    def __init__(self, nombre, mundos):
        self.nombre = nombre
        self.mundos = mundos
```

```
# =====
# 9. RESULTADOS DE PARTIDA (herencia simple)
# =====
```

```
class ResultadoPartida:
    def __init__(self, mensaje):
```

```
self.mensaje = mensaje
```

```
def mostrar_animacion(self):  
    print(f" >>> {self.mensaje} <<<")
```

```
class Ganaste(ResultadoPartida):  
    def __init__(self):  
        super().__init__("¡GANASTE!")  
  
    def mostrar_animacion(self):  
        super().mostrar_animacion()  
        print(" 🎉 Confeti y nuevo nivel/mundo desbloqueado.")
```

```
class Perdiste(ResultadoPartida):  
    def __init__(self):  
        super().__init__("PERDISTE")  
  
    def mostrar_animacion(self):  
        super().mostrar_animacion()  
        print(" 🔄 El nivel actual se reinicia.")
```

```
# =====  
# 10. JUGADOR (estado global vs. estado local)  
# =====
```

```
class Jugador:  
    def __init__(self, nombre):  
        self.nombre = nombre  
        self.registrado = False  
        # --- Variables GLOBALES del jugador (estado macro del juego) ---  
        self.puntaje_total = 0  
        self.monedas = 0  
        self.inventario_cosmeticos = []  
        self.herramientas = []  
        self.vehiculos = []  
        self.poderes = []  
        self.vehiculo_actual = None  
        # --- Variables LOCALES efímeras (solo válidas durante una carrera) ---  
        self.progreso = 0  
        self.velocidad_bonus = 0  
        self.escudo_activo = False  
  
    def registrar(self):  
        self.registrado = True  
        print(f" ✅ ¡Bienvenido, {self.nombre}! Cuenta registrada.")
```

```

def mostrar_inventario(self):
    print("\n--- 📦 INVENTARIO ---")
    print(f" Monedas Python: {self.monedas}")
    print(f" Vehículo actual: {self.vehiculo_actual.nombre if self.vehiculo_actual else 'Ninguno'}")
    print(" Vehículos:")
    for v in self.vehiculos:
        print(f"   - {v}")
    print(" Poderes:")
    for p in self.poderes:
        estado = " 🔒 " if p.bloqueado else " ✅ "
        print(f"   - {estado} {p.nombre} (dura {p.duracion}s)")
    print(" Herramientas:")
    for h in self.herramientas:
        estado = " ✅ " if h.disponible else " 🔒 "
        print(f"   - {estado} {h.nombre} (dura {h.duracion}s)")
    # Ciclo FOR para desglosar todos los premios/artículos acumulados
    print(" Artículos estéticos ganados:")
    for item in self.inventario_cosmeticos:
        print(f"   - 🏆 {item}")

```

```

def desglosar_premios(jugador, monedas_ganadas, item_ganado):
    """
    Ciclo FOR (requerimiento del análisis) para calcular y mostrar en
    pantalla todos los premios acumulados al finalizar la carrera.
    """
    premios = [("Monedas Python", monedas_ganadas), ("Artículo desbloqueado", item_ganado)]
    print("\n --- 📦 Desglose de premios ---")
    for concepto, valor in premios:
        print(f"   {concepto}: {valor}")

```

```

# =====
# 11. TOGGLES DE CONFIGURACIÓN Y CONFIRMACIONES
# =====

```

```

class ToggleConfiguracion:
    def __init__(self, nombre, activado=True):
        self.nombre = nombre
        self.activado = activado

    def alternar(self):
        respuesta = pedir_opcion(f" ¿Seguro que deseas cambiar '{self.nombre}'? (si/no): ",
                                  ["si", "no"])
        if respuesta == "si":
            self.activado = not self.activado
            estado = "activado" if self.activado else "desactivado"

```

```

        print(f" 🛠️ {self.nombre} {estado}.")
    else:
        print(" Cambio cancelado.")

```

```

class DecisionConfirmacion:
    def __init__(self, pregunta):
        self.pregunta = pregunta

    def confirmar(self):
        respuesta = pedir_opcion(f" {self.pregunta} (si/no): ", ["si", "no"])
        return respuesta == "si"

```

```

# =====
# 12. PANTALLAS DE MENÚ (herencia: mostrar/ocultar/volver_al_menu)
# =====

```

```

class PantallaMenu:
    def __init__(self, nombre):
        self.nombre = nombre

    def mostrar(self):
        print(f"\n===== {self.nombre.upper()} =====")

    def ocultar(self):
        pass # en consola no aplica, se deja por compatibilidad con el diagrama

    def volver_al_menu(self):
        print(" ⬅ Volviendo al menú principal...")

```

```

class Configuracion(PantallaMenu):
    def __init__(self):
        super().__init__("Configuración")
        self.efectos = ToggleConfiguracion("Efectos de sonido")
        self.musica = ToggleConfiguracion("Música")
        self.notificaciones = ToggleConfiguracion("Notificaciones")
        self.graficos = ToggleConfiguracion("Altos gráficos")

    def ejecutar(self):
        self.mostrar()
        opciones = {"1": self.efectos, "2": self.musica, "3": self.notificaciones,
                    "4": self.graficos}
        print(" 1) Efectos 2) Música 3) Notificaciones 4) Gráficos 5) Restablecer 0) Volver")
        op = pedir_opcion(" Elige una opción: ", list(opciones.keys()) + ["5", "0"])
        if op in opciones:
            opciones[op].alternar()

```

```

elif op == "5":
    if DecisionConfirmacion("¿Restablecer toda la configuración?").confirmar():
        for t in opciones.values():
            t.activado = True
        print("  Configuración restablecida.")
self.volver_al_menu()

```

```

class Tienda(PantallaMenu):

```

```

    def __init__(self, catalogo_herramientas, catalogo_vehiculos, catalogo_poderes):
        super().__init__("Tienda")
        self.catalogo_herramientas = catalogo_herramientas
        self.catalogo_vehiculos = catalogo_vehiculos
        self.catalogo_poderes = catalogo_poderes

```

```

    def ejecutar(self, jugador):
        self.mostrar()
        print(f" Monedas disponibles: {jugador.monedas}")
        print(" 1) Herramientas 2) Vehículos 3) Poderes 0) Volver")
        op = pedir_opcion(" Elige una categoría: ", ["1", "2", "3", "0"])
        if op == "1":
            self._comprar(jugador, self.catalogo_herramientas, "precio_fijo", jugador.herramientas)
        elif op == "2":
            self._comprar(jugador, self.catalogo_vehiculos, "precio", jugador.vehiculos)
        elif op == "3":
            self._comprar(jugador, self.catalogo_poderes, "costo", jugador.poderes)
        self.volver_al_menu()

```

```

    def _comprar(self, jugador, catalogo, attr_precio, inventario_jugador):
        for i, articulo in enumerate(catalogo):
            precio = 100 if attr_precio == "precio_fijo" else getattr(articulo, attr_precio)
            print(f" [{i}] {articulo.nombre} - {precio} monedas")
        op = pedir_opcion(" Escribe el número a comprar (o 'x' para cancelar): ",
                        [str(i) for i in range(len(catalogo))] + ["x"])
        if op == "x" or op is None:
            return
        articulo = catalogo[int(op)]
        precio = 100 if attr_precio == "precio_fijo" else getattr(articulo, attr_precio)
        if jugador.monedas >= precio:
            jugador.monedas -= precio
            if hasattr(articulo, "desbloquear"):
                articulo.desbloquear()
            else:
                articulo.disponible = True
            if articulo not in inventario_jugador:
                inventario_jugador.append(articulo)
            print(f"  Compraste {articulo.nombre}.")
        else:

```

```
print(" ❌ No tienes suficientes monedas.")
```

```
# =====  
# 13. CLASE PRINCIPAL DEL JUEGO  
# =====
```

```
class LineaDeFuego:
```

```
    """
```

```
    Orquesta el flujo completo según el DIAGRAMA DE FLUJO:
```

```
    Línea de Fuego -> Registrarse -> Pantalla de Inicio -> Menú ->
```

```
    Jugar -> Mapas -> Mundos -> Niveles -> Ganaste/Perdiste -> Premio
```

```
    """
```

```
    def __init__(self):
```

```
        self.jugador = None
```

```
        self.niveles = crear_niveles()
```

```
        self.vehiculos = crear_vehiculos()
```

```
        self.poderes = crear_poderes()
```

```
        self.herramientas = crear_herramientas_tienda()
```

```
        self.jefes = {"Mundo 1": TurboElMono(), "Mundo 2": VoltioBot(), "Mundo 3": CapitanNova()}
```

```
        mundo1 = Mundo("Mundo 1: Isla", self.niveles[0:2], self.jefes["Mundo 1"], bloqueado=False)
```

```
        mundo2 = Mundo("Mundo 2: Metrópolis", self.niveles[2:4], self.jefes["Mundo 2"])
```

```
        mundo3 = Mundo("Mundo 3: Cosmos", self.niveles[4:6], self.jefes["Mundo 3"])
```

```
        self.mundos = [mundo1, mundo2, mundo3]
```

```
        self.mapa1 = Mapa("Mapa 1", [mundo1])
```

```
        self.mapa2 = Mapa("Mapa 2", [mundo2, mundo3])
```

```
        self.mapas = [self.mapa1, self.mapa2]
```

```
        self.configuracion = Configuracion()
```

```
        self.tienda = Tienda(self.herramientas, self.vehiculos, self.poderes)
```

```
# ----- PANTALLAS INICIALES -----
```

```
def pantalla_linea_de_fuego(self):
```

```
    print("\n#####")
```

```
    print("#      LÍNEA DE FUEGO  🚒      #")
```

```
    print("#####")
```

```
    op = pedir_opcion("¿Deseas registrarte? (si/no): ", ["si", "no"])
```

```
    if op == "si":
```

```
        nombre = input("Ingresa tu nombre de piloto: ").strip() or "Piloto"
```

```
        self.jugador = Jugador(nombre)
```

```
        self.jugador.registrar()
```

```
        self.jugador.vehiculo_actual = self.vehiculos[0]
```

```
        self.jugador.vehiculos.append(self.vehiculos[0])
```

```
        self.jugador.herramientas.append(self.herramientas[0])
```

```

        self.pantalla_inicio()
    else:
        print("Regresando a registrarse (obligatorio para continuar)...")
        self.pantalla_linea_de_fuego()

def pantalla_inicio(self):
    print(f"\n=== PANTALLA DE INICIO - Bienvenido, {self.jugador.nombre} ===")
    input("Presiona ENTER para iniciar juego...")
    self.menu_principal()

# ----- MENÚ PRINCIPAL -----
def menu_principal(self):
    while True:
        print("\n===== MENÚ =====")
        print("1) Jugar")
        print("2) Mapas")
        print("3) Configuración")
        print("4) Tienda")
        print("5) Ver inventario")
        print("6) Salir")
        op = pedir_opcion("Elige una opción: ", ["1", "2", "3", "4", "5", "6"])
        if op is None:
            continue
        if op == "1" or op == "2":
            self.pantalla_mapas()
        elif op == "3":
            self.configuracion.ejecutar()
        elif op == "4":
            self.tienda.ejecutar(self.jugador)
        elif op == "5":
            self.jugador.mostrar_inventario()
        elif op == "6":
            if DecisionConfirmacion("¿Está seguro que desea salir?").confirmar():
                print("👋 ¡Gracias por jugar Línea de Fuego!")
            return

# ----- MAPAS / MUNDOS / NIVELES -----
def pantalla_mapas(self):
    print("\n===== MAPAS =====")
    for i, mapa in enumerate(self.mapas):
        print(f" [{i}] {mapa.nombre}")
    op = pedir_opcion("Elige un mapa (o 'x' para volver): ",
                      [str(i) for i in range(len(self.mapas))] + ["x"])
    if op == "x" or op is None:
        return
    mapa = self.mapas[int(op)]
    self.seleccionar_mundo(mapa)

```

```

def seleccionar_mundo(self, mapa):
    print(f"\n--- {mapa.nombre}: Mundos disponibles ---")
    for i, mundo in enumerate(mapa.mundos):
        estado = "🔒 BLOQUEADO" if mundo.bloqueado else "✅ DISPONIBLE"
        print(f" [{i}] {mundo.nombre} - {estado}")
    op = pedir_opcion("Elige un mundo (o 'x' para volver): ",
                      [str(i) for i in range(len(mapa.mundos))] + ["x"])
    if op == "x" or op is None:
        return
    mundo = mapa.mundos[int(op)]
    if mundo.bloqueado:
        print("🔒 Este mundo está bloqueado. Completa el mundo anterior para pasar.")
        return
    self.seleccionar_nivel(mundo)

def seleccionar_nivel(self, mundo):
    print(f"\n--- {mundo.nombre}: Niveles --- Jefe: {mundo.jefe}")
    for i, nivel in enumerate(mundo.niveles):
        estado = "🔒 BLOQUEADO" if nivel.bloqueado else ("🏆 COMPLETO" if nivel.completado
else "▶ DISPONIBLE")
        print(f" [{i}] {nivel.nombre} (Dif. {nivel.dificultad}) - {estado}")
    op = pedir_opcion("Elige un nivel (o 'x' para volver): ",
                      [str(i) for i in range(len(mundo.niveles))] + ["x"])
    if op == "x" or op is None:
        return
    nivel = mundo.niveles[int(op)]
    if nivel.bloqueado:
        print("🔒 Nivel bloqueado. Termina el nivel anterior primero.")
        return
    self.jugar_nivel(nivel, mundo)

def jugar_nivel(self, nivel, mundo):
    gano = nivel.jugar(self.jugador, mundo.jefe)
    indice_global = self.niveles.index(nivel)
    if gano:
        Ganaste().mostrar_animacion()
        desglosar_premios(self.jugador, nivel.premio_monedas, nivel.premio_item)
        desbloquear_siguiente_nivel(self.niveles, indice_global, self.jugador)
        for m in self.mundos:
            m.verificar_desbloqueo(self.jugador)
        if all(n.completado for n in self.niveles):
            print(f"\n🏁🎉 ¡TERMINASTE EL JUEGO! Has completado todos los niveles. 🎉🏁")
    else:
        Perdiste().mostrar_animacion()
        op = pedir_opcion("¿Reiniciar el nivel? (si/no): ", ["si", "no"])
        if op == "si":
            self.jugar_nivel(nivel, mundo)

```

```
# =====  
# 14. PUNTO DE ENTRADA  
# =====
```

```
def main():  
    try:  
        juego = LineaDeFuego()  
        juego.pantalla_linea_de_fuego()  
    except Exception as error:  
        # Última barrera de seguridad: nunca cerrar el juego de forma abrupta  
        print(f"\n△ Ocurrió un error inesperado, pero el juego se cerrará con seguridad: {error}")  
  
if __name__ == "__main__":  
    main()
```